

Adhikar — AI for Civic and Legal Empowerment

System Architecture, Build Plan & Agent Build Prompts — OOSC 4.0 Hackathon

Prepared for: Yan — Team of 2-3

August 2026

Contents

1. Project Brief	1
2. Core Design Principle: Filter → Retrieve → Verify → Cite	2
3. Shared Core Components	2
4. Module Specifications	3
5. Tech Stack & Repository Structure	4
6. Architecture Decision Records (ADRs)	4
7. Build Prompts (for the Antigravity build agent)	6
8. Suggested Build Order & Ownership (2-3 people)	11

1. Project Brief

Problem Statement: AI for Civic and Legal Empowerment (Problem Statement 3, OOSC 4.0).

One-line pitch: A single AI platform that reads through bureaucratic and legal complexity on a citizen's behalf, and turns it into a concrete document, answer, or next step — in plain language.

The problem statement lists four illustrative directions. This build treats all four not as separate products, but as four **feature modules sitting on one shared core** — a single conversational profile engine, one retrieval-and-guardrail pattern, one frontend shell:

Module	Maps to problem statement direction
A — Scheme Eligibility Reader (MVP)	“Scheme Eligibility Reader — reads government portals and answers eligibility questions in plain language”
B — Application Auto-Filler	“Conversational Form-Filler — interviews the user and auto-populates the official form”

Module	Maps to problem statement direction
C — RTI Drafting Agent (stretch)	“RTI Drafting Agent — converts a plain-language question into a properly formatted application to the right department”
D — Rights Navigator (stretch)	“Rights Navigator — explains, in simple terms, what a person can do about a specific tenant, consumer or workplace dispute”

This satisfies “open reinterpretation of bureaucracy translation is welcome” honestly — the platform’s actual product is the **translation layer itself** (bureaucratic text → plain language → grounded answer → filled document), and Modules A-D are four applications of that one layer.

Build honesty note: Module A + B together are the full MVP and must be demo-solid on their own. C and D are additive — the architecture is modular specifically so the team can stop after B and still have a complete, working demo if time runs out. Do not let C or D block A or B.

2. Core Design Principle: Filter → Retrieve → Verify → Cite

Every module in this system that makes a factual or legal claim follows the **same three-stage pattern**, because the single biggest risk in this domain is a confident, wrong answer about someone’s rights or eligibility.

1. **Deterministic filter** — narrow the space using structured data (no LLM, cannot hallucinate).
2. **Grounded retrieval** — pull the exact source clause/text relevant to the narrowed candidates (RAG).
3. **Constrained verification** — LLM is only allowed to answer using the retrieved text, must cite it, and must answer UNSURE rather than guess when the text doesn’t clearly support a verdict.

Every module below is a specific application of this same pattern. Building it once, as a shared library, and reusing it everywhere is the single most important architectural decision in this project — it is also the subject of ADR-001 (see Section 6).

3. Shared Core Components

These are built once and used by every module.

3.1 Conversational Profile Engine

A multi-turn interview component that extracts structured slots from free-text user input (age, location/state, occupation, income bracket, category, situation description) and asks a follow-up only when a slot needed by the active module is missing. This is the “Conversational Form-Filler” capability from the problem statement — it is not module-specific, it is the shared interviewing brain that Modules A, B, and C all call into.

3.2 Retrieval & Guardrail Library

A shared backend library implementing the Filter → Retrieve → Verify → Cite pattern generically, parameterized by: - which structured filter fields to use - which vector collection to retrieve from - what counts as a valid citation for that domain

Modules A, C, and D all import this library rather than reimplementing retrieval logic.

3.3 Document Generator

Takes a template (scheme application format, RTI format) plus a filled profile and produces a downloadable, properly formatted document. Used by Module B and Module C.

3.4 Frontend Shell

One dashboard: left panel is the conversation, right panel switches between “Matched Schemes,” “Your Rights,” and “Generated Document” views depending on which module the conversation is currently in. One shell, four content modes — not four separate apps.

4. Module Specifications

Module A — Scheme Eligibility Reader (MVP, build first)

Data source: MyScheme government dataset (structured; eligibility, benefits, application process, source link already present — no scraping needed). **Flow:** 1. Profile Engine collects user situation. 2. Deterministic filter narrows ~700+ schemes to a short candidate list using structured slots (age, occupation, state, income, category). 3. Retrieval & Guardrail library verifies each candidate against its actual eligibility clause; returns ELIGIBLE / NOT ELIGIBLE / UNSURE with the cited clause. 4. Results panel shows ranked matches, each with the source clause visible — this traceability is the trust feature judges will notice.

Module B — Application Auto-Filler

Depends on: Module A output + Document Generator. **Flow:** user selects a matched scheme → the scheme’s own `application_process` field (already in the dataset) is used as the template → Document Generator fills it from the already-collected profile → downloadable document. This is deliberately *not* a new capability — it is the Document Generator applied to Module A’s output, which is why it’s cheap to add once A exists.

Module C — RTI Drafting Agent (stretch)

The real risk in this module is department routing, not drafting. Do not attempt free-form department classification from scratch. **Flow:** 1. User describes their grievance in plain language. 2. LLM classifies against a **curated, static directory** of common RTI-relevant departments/public authorities (built ahead of time, not scraped live) and returns a routing suggestion with a confidence score. 3. If confidence is below a set threshold, the system **asks the user to confirm or pick from 2-3 candidate departments** rather than silently guessing — human-in-the-loop, not full automation. 4. Once confirmed, Document Generator produces a properly formatted RTI application to that department using the profile already collected.

Module D — Rights Navigator (stretch)

Flow: 1. User describes a tenant, consumer, or workplace dispute. 2. Retrieval & Guardrail library retrieves from a **curated knowledge base** of plain-language rights explainers (built from public legal-aid sources, the Consumer Protection Act, model tenancy laws, labour law basics — pre-processed and embedded ahead of time, same as Module A's scheme data). 3. LLM explains the relevant right/remedy in plain language, citing the source explainer, or returns UNSURE and suggests legal aid contact info if nothing in the knowledge base clearly applies.

5. Tech Stack & Repository Structure

Stack: React + Tailwind (frontend) · FastAPI (backend) · ChromaDB (vector store, local) · Gemini as primary LLM, Claude API as fallback · Bhashini as an optional, non-blocking pre/post-processing toggle for speech and translation — never on the critical path of any module.

```
/backend
  /core
    profile_engine.py          # Section 3.1
    retrieval_guardrail.py      # Section 3.2 – shared library
    document_generator.py      # Section 3.3
  /modules
    /scheme_matcher            # Module A
    /application_filler        # Module B
    /rti_drafting               # Module C
    /rights_navigator          # Module D
  /ingestion
    myscheme_ingest.py
    rights_kb_ingest.py
    rti_departments.json
  /api
    main.py
/frontend
  /src
    /components
    /views                     # Matched Schemes / Your Rights / Generated Document
/data
  /myscheme_processed
  /rights_kb_processed
/docs
  /adr
    0000-adr-template.md
README.md
```

6. Architecture Decision Records (ADRs)

Every module in this project involves at least one decision with real trade-offs (accuracy vs. speed, automation vs. human-in-the-loop, build-vs-reuse). These decisions get written down as they're made, not reconstructed afterward — this is both good engineering practice

and something a hackathon judging panel will specifically respect if they ask “why did you build it this way.”

Rule for the build agent: every prompt in Section 7 requires writing one ADR file in /docs/adr/ before or alongside the implementation it covers. Number them sequentially as they’re created.

ADR Template (/docs/adr/0000-adr-template.md)

ADR-XXX: <Short decision title>

Date: <YYYY-MM-DD>

Status: Accepted

Context

What problem or choice prompted this decision? What constraints applied?

Decision

What was decided, stated plainly in 2-4 sentences.

Alternatives Considered

- Option 1 – why not chosen
- Option 2 – why not chosen

Consequences

What this makes easier, what it makes harder, what risk it introduces or removes.

Planned ADR Log

ID	Title	Module
ADR-001	Filter-then-verify over pure	Core
ADR-002	RAG for factual/legal claims	Module A
ADR-003	MyScheme data ingestion & structured schema design	Core
ADR-004	Vector store & embedding model choice	Core
ADR-005	Conversational profile engine slot design	Core
ADR-006	Guardrail caveat schema (conditional/state-dependent rights)	Core
ADR-007	(reserved, contingent) New decision surfaced while building Module A	Module A
ADR-008	Document generation approach (template-fill vs. free generation)	Module B
ADR-009	RTI department routing: human-in-the-loop over full automation	Module C
ADR-010	Rights Navigator knowledge base sourcing & curation	Module D

ID	Title	Module
ADR-010	Frontend state model (single shell, switchable views)	Frontend
ADR-011	Bhashini as non-blocking toggle, not critical path	Integration

7. Build Prompts (for the Antigravity build agent)

How to use these: paste one prompt at a time into Antigravity, in order. Each prompt is scoped to one deliverable and ends with an explicit ADR requirement so the decision trail builds up alongside the code. Do not skip the ADR step — it is part of the deliverable, not optional documentation.

Prompt 0 — Project Scaffolding

You are acting as a senior full-stack engineer setting up a new project.

Project: "Adhikar" — an AI civic/legal empowerment platform for the OOSC 4.0 hackathon (Problem Statement 3: AI for Civic and Legal Empowerment).

Set up the repository with this exact structure:

```
/backend/core, /backend/modules, /backend/ingestion, /backend/api
/frontend/src/components, /frontend/src/views
/data/myscheme_processed, /data/rights_kb_processed
/docs/adr
```

Stack: FastAPI backend, React + Tailwind frontend, ChromaDB as local vector store, Gemini as primary LLM client with a swappable interface so Claude can be used as a fallback. No hosted database — everything must run locally for a hackathon demo with no external infra dependency beyond the LLM API and an optional Bhashini call.

Create /docs/adr/0000-adr-template.md with this exact template:

```
# ADR-XXX: <Short decision title>
Date: <YYYY-MM-DD>
Status: Accepted
## Context
## Decision
## Alternatives Considered
## Consequences
```

Then create /docs/adr/0001-filter-then-verify-pattern.md as ADR-001, recording the decision to use a three-stage Filter -> Retrieve -> Verify pattern for every module that makes a factual or legal claim, instead of raw free-generation RAG, because unverified LLM claims about eligibility or legal rights are the single biggest credibility risk in this domain. Document the

alternative considered (plain RAG with no deterministic filter stage) and why it was rejected (higher hallucination surface, no traceability for citations).

Do not implement any feature logic yet – this prompt is scaffolding and ADR-001 only. Confirm the structure with a tree output when done.

Prompt 1 — Data Ingestion (Module A foundation)

You are the backend engineer on the Adhikar project. Context: the repo scaffolding and ADR-001 already exist – read /docs/adr/0001-filter-then-verify-pattern.md before starting so your implementation matches that decision.

Task: build /backend/ingestion/myscheme_ingest.py to process the MyScheme government schemes dataset into two outputs:

1. A structured JSON/CSV with one row per scheme:
`{id, name, eligibility_text, benefits, application_process, source_url, structured_slots: {age_min, age_max, occupation, state, income_max, category}}`
The structured_slots fields should be extracted from eligibility_text via one LLM pass per scheme (batch this, don't call per-request later).
2. Embeddings of eligibility_text chunks (one chunk per scheme, tagged with scheme_id) stored in a local ChromaDB collection called "scheme_eligibility".

Write the processed outputs to /data/myscheme_processed/.

Before finishing, write /docs/adr/0002-myscheme-ingestion-schema.md as ADR-002, recording why structured_slots are extracted as a one-time batch LLM pass (not at query time) and why eligibility_text is chunked per-scheme rather than by paragraph. Note the alternative of skipping structured_slots entirely and relying only on RAG, and why that was rejected (no deterministic filter stage possible without it, which violates ADR-001).

Prompt 2 — Shared Core: Profile Engine + Retrieval/Guardrail Library

You are the backend engineer on Adhikar. Read ADR-001 and ADR-002 first.

Task 1: build /backend/core/profile_engine.py – a conversational slot-filling component. Given free-text user input plus a list of "required slots" passed in by whichever module calls it (e.g. age, state, occupation, income, category, or a free-text dispute description), it should extract what it can via LLM, track what's still missing, and generate one natural follow-up question at a time for missing required slots. It must be module-agnostic – Module A, B, and C will all call this with different required-slot lists.

The "state" slot specifically must be normalized against a fixed list of India's states and union territories (not free text) before being written to the profile – downstream modules depend on an exact match: Module C's RTI routing needs it to pick between central and state jurisdiction, and Module D's tenant-rights answers need it to check Model Tenancy Act adoption status.

Reject/re-prompt if the extracted state can't be confidently mapped to the fixed list rather than storing an ambiguous value.

Task 2: build `/backend/core/retrieval_guardrail.py` – a generic library implementing the Filter -> Retrieve -> Verify -> Cite pattern from ADR-001. It should accept: a structured filter function, a Chroma collection name, and a "verification prompt template" that forces the LLM to output one of ELIGIBLE / NOT_ELIGIBLE / UNSURE (or a domain-appropriate equivalent) plus the exact cited source text, and never allows an answer without a citation or without UNSURE when the retrieved text doesn't clearly support a verdict.

The output schema must also support an optional "caveat" field, separate from the verdict, for cases where the source is clear but the answer depends on a condition outside the retrieved text (e.g. "this right applies only if your state has adopted the Model Tenancy Act"). When a retrieved chunk is tagged with a caveat at ingestion time, the library must always surface it alongside the verdict – never suppress it, and never resolve it silently on the LLM's own judgment. This is required for Module D, but build it into the generic library now rather than special-casing it later.

Write `/docs/adr/0003-vector-store-and-embeddings.md` as ADR-003 recording the choice of local ChromaDB over a hosted vector DB (Pinecone/Weaviate) for this build – reasoning: zero setup time, no network dependency during the demo, dataset is small enough (~700 schemes) that local is sufficient – and the embedding model chosen.

Write `/docs/adr/0004-profile-engine-slot-design.md` as ADR-004 recording why the profile engine is built as one shared, module-agnostic component instead of separate per-module interview logic, what slot schema was settled on, and why the state slot specifically is normalized against a fixed list rather than stored as free text.

Write `/docs/adr/0005-guardrail-caveat-schema.md` as ADR-005 recording the decision to extend the citation schema with an optional caveat field instead of treating conditional rights (e.g. state-dependent tenancy law) as either a plain ELIGIBLE verdict or a forced UNSURE. Record the alternative considered (handling caveats ad hoc inside Module D only) and why building it into the shared library now was preferred – avoids each future module reinventing the same conditional-answer handling.

Prompt 3 — Module A: Scheme Eligibility Reader

You are the backend + frontend engineer on Adhikar. Read ADR-001 through ADR-005 before starting.

Task: implement `/backend/modules/scheme_matcher/` using `profile_engine.py` to collect the user's situation and `retrieval_guardrail.py` (pointed at the "scheme_eligibility" Chroma collection and the structured_slots filter) to produce a ranked list of matched schemes, each with an ELIGIBLE/NOT_ELIGIBLE/UNSURE verdict and its cited clause.

Expose this as a FastAPI endpoint in `/backend/api/main.py`.

Build the corresponding frontend view in /frontend/src/views – a "Matched Schemes" panel showing scheme name, verdict, benefits, and the cited clause text expandable per card.

This module is the MVP. It must work end-to-end and be demo-stable before any other module is started.

No new ADR is required for this prompt – it is a direct application of ADR-001 through ADR-005, not a new decision. If you find yourself making a new non-obvious choice while implementing this, stop and write it as ADR-006 before continuing, using the template in /docs/adr/0000-adr-template.md. If no such decision comes up, ADR-006 is simply skipped – do not renumber later ADRs to fill the gap.

Prompt 4 — Module B: Application Auto-Filler + Document Generator

You are the backend + frontend engineer on Adhikar. Module A is complete and demo-stable – do not modify its core matching logic in this prompt.

Task: build /backend/core/document_generator.py – takes a template string (here, a matched scheme's application_process field) and a filled user profile, and produces a formatted, downloadable document (PDF or DOCX) with the profile fields correctly placed.

Wire this into /backend/modules/application_filler/ – triggered when a user selects a matched scheme from Module A's results. Add a "Generate Application" button to the frontend results view and a document preview/download panel.

Write /docs/adr/0007-document-generation-approach.md as ADR-007, recording the decision to template-fill from the scheme's own application_process data rather than free-generating the document text from scratch with the LLM, and why (accuracy – the format is already correct and government-sourced; free generation risks an incorrectly formatted or invented application).

Prompt 5 — Module C: RTI Drafting Agent (stretch)

You are the backend + frontend engineer on Adhikar. Modules A and B are complete and demo-stable. This module is additive – if you hit a blocker, stop and report back rather than touching A or B's code.

Task: build /backend/ingestion/rti_departments.json – a curated static list of common RTI-relevant public authorities/departments (do not attempt to scrape or auto-discover this list; hand-curate a reasonable set for the demo scope, e.g. municipal, state welfare, PDS/ration, land records, education).

Build /backend/modules/rti_drafting/ using profile_engine.py to collect the grievance description, an LLM classification step against rti_departments.json that returns a routing suggestion with a confidence score, and a human-in-the-loop confirmation step: if confidence is below a threshold you define, present 2-3 candidate departments to the user in the

frontend and let them pick, rather than routing automatically. Once confirmed, call `document_generator.py` to produce a formatted RTI application.

Write `/docs/adr/0008-rti-department-routing.md` as ADR-008, recording the decision to use human-in-the-loop confirmation instead of fully automatic department routing, and why: incorrect automatic routing produces a useless application with no way for the user to catch the error, whereas a low-confidence confirmation step costs one extra click and eliminates that failure mode. Record the confidence threshold chosen and how it was decided.

Prompt 6 — Module D: Rights Navigator (stretch)

You are the backend + frontend engineer on Adhikar. Modules A and B are complete and demo-stable; treat this as additive, same as Module C.

Task: build `/backend/ingestion/rights_kb_ingest.py` to process a curated set of plain-language rights explainers (tenant rights, consumer protection, workplace disputes – sourced from public legal-aid materials, not scraped live) into a Chroma collection called "rights_kb", following the same chunking/embedding pattern as ADR-002/ADR-003.

Build `/backend/modules/rights_navigator/` using `profile_engine.py` to collect the dispute description and `retrieval_guardrail.py` (pointed at "rights_kb") to retrieve and cite the relevant explainer, returning a plain-language explanation with citation, or UNSURE plus a suggestion to contact legal aid if nothing in the knowledge base clearly applies.

Add a "Your Rights" view to the frontend showing the explanation and its source citation.

Write `/docs/adr/0009-rights-kb-sourcing.md` as ADR-009, recording exactly which sources were curated for the rights knowledge base, why those sources were chosen over others, and the explicit limitation that this module only covers the scope of its curated knowledge base and must return UNSURE outside that scope rather than reasoning freely.

Prompt 7 — Frontend Shell + Bhashini Integration (polish pass)

You are the frontend engineer on Adhikar, doing the final integration pass.

Task 1: unify the frontend into one dashboard shell – left panel is the ongoing conversation (shared across all modules via `profile_engine.py`'s session state), right panel switches between "Matched Schemes," "Your Rights," and "Generated Document" views depending on which module is active in the conversation. Do not build separate apps per module.

Task 2: add Bhashini as an optional toggle for speech input/output and translation, implemented strictly as a pre/post-processing wrapper around the existing text pipeline – it must not be called from inside any module's core logic, and disabling the toggle must leave every module working identically in text-only English.

Write `/docs/adr/0010-frontend-shell-state-model.md` as ADR-010, recording the decision to use one shell with switchable views over separate per-module frontends, and why (shared conversation state, one profile collected once, reused across modules).

Write `/docs/adr/0011-bhashini-non-blocking-toggle.md` as ADR-011, recording the decision to keep Bhashini strictly outside the critical path, and the fallback behavior when it is disabled or fails.

8. Suggested Build Order & Ownership (2-3 people)

Phase	Owner	Prompts
Scaffolding + data ingestion	Person A	Prompt 0, 1
Shared core (profile engine + guardrail library)	Person A	Prompt 2
Module A (Scheme Reader)	Person A + B	Prompt 3
Module B (Application Filler)	Person B	Prompt 4
Module C (RTI Drafting) — stretch	Person C (if 3-person team)	Prompt 5
Module D (Rights Navigator) — stretch	Person C (if 3-person team)	Prompt 6
Frontend shell + Bhashini polish	Person B	Prompt 7

If the team is 2 people, stop after Prompt 4 (A + B complete) and only attempt Prompt 5/6 with remaining time — a solid, demo-stable MVP beats two half-finished stretch modules.